

Hardware Ray Tracing for Bounded Decision-Path Scoring: An OptiX Backend Primitive

GAFIME Project
Technical Disclosure Draft

July 2026

Abstract

Tree ensembles and tree-derived decision-path features are among the most effective tools for tabular learning, but large-scale candidate scoring remains limited by candidate-row predicate evaluation, intermediate membership materialization, and reduction of binary indicators against a target. This paper describes an experimental CUDA backend primitive that maps bounded, shallow gradient-boosted decision-tree (GBDT) path regions onto NVIDIA hardware ray tracing using OPTiX. A decision path over one to three numerical features is represented as an axis-aligned region, a tabular row is represented as a point, and path membership becomes a point-containment query. The low-level ABI keeps path descriptors compact and returns one result row per path with metric columns. It is not yet the execution path of the public GAFIME Python decision-path adapter, which still expands membership before invoking the generic continuous engine. The measurements therefore characterize a backend primitive, not end-to-end GBDT training, inference, or public-API acceleration.

The main performance contribution is not faster materialization of a dense row-by-path membership matrix. Instead, traversal results are reduced on device into compact per-path statistics and exported through the experimental result ABI. Traversal is hybrid: OPTiX performs BVH traversal and intersection work, while programmable CUDA code performs exact guards, accumulation, and metric finalization. For tree-leaf-like batches whose regions are finite, bounded, two-dimensional, and non-overlapping within each group, a first-hit traversal mode terminates after the first exact in-region hit and fails closed if the non-overlap proof is unavailable. On an RTX 4060 Laptop GPU, a $262,144 \times 8,192$ case takes 56.109 ms on the first call and 0.886 ms at resident warm p50 over five samples. The latter is 2.424 trillion membership-equivalent evaluations/s, but the end-to-end effective ray rate is 2.367 billion rays/s over eight groups; these are different denominators. Maximum absolute score difference is 1.19209×10^{-7} against an exhaustive CPU reference. A $262,144 \times 1,048,576$ case takes 467.746 ms first-call and 20.180 ms resident warm p50, or 13.621 trillion membership-equivalent evaluations/s and 0.104 billion end-to-end effective rays/s. A linear partition oracle validates that larger result to 5.58794×10^{-9} . These results identify a narrow systems primitive: bounded decision-path scoring can benefit from BVH traversal and fused reduction when geometry is tree-like and resident reuse is available.

Keywords: GBDT systems; RT cores; OptiX; candidate scoring; fused reduction; tabular learning.

1 Introduction

Gradient boosting decision trees are a standard workhorse for tabular machine learning. Friedman formulated gradient boosting as greedy function approximation [1]; XGBoost [2], LightGBM [3], and CatBoost [4] showed that practical performance depends as much on systems decisions as on

statistical modeling. Cache-aware layout, sparsity handling, histogram construction, categorical encoding, and parallelism choices decide whether a tree system scales.

This work targets a related but distinct problem: scoring very large numbers of tree-derived decision-path candidates as explicit features. The intended GAFIME representation is a compact expression or path descriptor accompanied by computed metrics, rather than a candidate-expanded vector. The experimental CUDA ABI follows that representation. The current public Python decision-path adapter does not: it materializes each membership vector and feeds an expanded matrix to the generic continuous engine. This disclosure therefore isolates the backend primitive and the integration contract that a future public path must satisfy. The scale problem is candidate-row work, not ordinary matrix multiplication.

A shallow decision path is a conjunction of threshold predicates. For example,

$$x_{f_0} > a \quad \wedge \quad x_{f_1} \leq b.$$

For one, two, or three participating features, this conjunction is naturally an interval, rectangle, or box. Hardware ray tracing units are specialized to traverse spatial acceleration structures and answer geometric intersection queries at high throughput. NVIDIA OPTIX provides a programmable ray tracing system for GPUs [15], and RTX-class GPUs include dedicated ray traversal and intersection hardware [16]. This suggests the following mapping:

$$\text{row} \longrightarrow \text{point}, \quad \text{decision path} \longrightarrow \text{bounded region}.$$

The mapping is useful only if three engineering constraints hold. First, the geometric result must preserve the original predicate semantics, including strict and non-strict inequalities. Second, the output must remain compact; a fast path that writes a dense membership matrix moves the bottleneck rather than removing it. Third, backend ownership must remain explicit. In the target architecture Rust owns candidate semantics, scheduling, and fallback policy; CUDA executes only a validated request. The benchmark used here calls the low-level C ABI directly and does not establish that this ownership path is wired through the public API.

This paper is a technical disclosure of the GAFIME v1 CUDA RT-core implementation. It makes the following contributions:

1. It formalizes a geometric mapping for finite, bounded, low-dimensional GBDT path regions.
2. It describes an experimental CUDA C ABI that scores decision paths through OPTIX and exports one compact row per path with metric columns.
3. It introduces a fail-closed first-hit mode for non-overlapping tree-leaf-like regions, together with the equivalence condition that makes first-hit traversal semantically valid.
4. It reports first-call and resident warm performance on a laptop Ada GPU, with exhaustive and partition-oracle correctness checks, end-to-end effective ray rates, and membership-equivalent work rates reported separately.

2 Background and Related Work

2.1 GBDT Systems

XGBoost combines gradient tree boosting with a sparsity-aware split finder, weighted quantile sketching, and cache-conscious execution [2]. LightGBM improves histogram-based training

through gradient-based one-side sampling and exclusive feature bundling [3]. CatBoost addresses target leakage and categorical features through ordered boosting and specialized categorical handling [4]. GPU-oriented tree systems such as ThunderGBM [5] and GPU histogram training work [6] show that tree workloads can benefit from GPU parallelism when memory access, histogram conflicts, and algorithm structure are matched to the device.

The work here is not a replacement for these training systems. It focuses on candidate scoring for decision-path-derived features. That distinction matters: training chooses splits and updates an ensemble, while GAFIME evaluates a large candidate set and keeps compact scores. The relevant unit of work is a membership-equivalent candidate-row evaluation rather than a tree-training iteration.

2.2 Tree Inference and Compilers

Tree deployment systems such as Treelite [7] and optimized forest inference paths such as NVIDIA FIL [8] target fast prediction for already-trained forests. Compiler approaches for tree inference, including Treebeard [9], lower tree traversal into optimized code for a selected target. These systems optimize inference of model outputs. GAFIME instead scores candidate path features against a target and therefore needs compact reductions over path membership rather than final ensemble prediction alone.

2.3 Ray Tracing Systems

OPTIX is a general-purpose programmable ray tracing engine [15]. Dedicated RTX hardware exposes fixed-function traversal and intersection acceleration [16]. GPU ray traversal work has studied how traversal efficiency depends on scheduling, memory locality, divergence, and scene structure [17, 18]. Embree [19] demonstrated that carefully engineered ray traversal kernels can be useful reusable systems infrastructure on CPUs. This paper applies ray traversal to a tabular candidate-scoring problem: not rendering, but evaluating whether row-points belong to many decision-path regions.

The closest methodological precedent is non-rendering use of RT hardware. Zellmann et al. map scientific point-containment queries to zero-length rays [10]. RTIndex maps point and range lookups to GPU ray tracing [11], RTXRMQ maps range-minimum queries to closest-hit geometry [12], and RTScan maps conjunctive database scans into balanced 3D ray-tracing jobs [13]. These systems establish that a geometric reformulation can change both hardware use and algorithmic work. They also motivate a skeptical baseline: RasterScan retains a related grid data model but replaces BVH traversal and geometric intersection with rasterization and arithmetic comparisons, outperforming its RT-based database-indexing baselines in that domain [14]. That result is not a direct measurement of decision-path scoring, but it shows why comparisons against equally structure-aware non-RT algorithms are required.

3 Decision Paths as Regions

Let $X \in \mathbb{R}^{n \times d}$ be a tabular matrix with n rows and d features, and let $y \in \mathbb{R}^n$ be a target. A path candidate p is a conjunction of threshold terms:

$$p = \{(f_\ell, s_\ell, t_\ell)\}_{\ell=1}^L,$$

GBDT path	: $x_{f_0} > a \wedge x_{f_1} \leq b$
Region	: $(a, \infty) \times (-\infty, b]$
Row	: $q_i = (X_{i,f_0}, X_{i,f_1})$
Score	: $\rho(z_p, y)$ or $R^2(z_p, y)$
candidate descriptors $\rightarrow$ RT traversal $\rightarrow$ on-device counts/sums $\rightarrow$ compact result rows.	

Figure 1: Decision-path scoring as point-in-region traversal followed by compact on-device reduction. The experimental compact-score ABI does not export a dense row-by-path matrix.

where f_ℓ is a feature id, $s_\ell \in \{\leq, >\}$ is the comparison, and t_ℓ is a threshold. Membership for row i is

$$z_{p,i} = \begin{cases} 1, & \text{if all terms in } p \text{ hold for row } i, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 1 (Representable RT path). *A path batch is representable by the current RT backend when every threshold is finite and the entire resident feature matrix recorded at upload is finite. The implementation stores one matrix-wide finiteness bit, so a NaN or infinity in an unreferenced column still makes RT unsupported. A membership batch uses at most three feature axes in total; compact scoring may split paths into internal groups, each using at most three axes. Bounded two-dimensional groups may use triangle geometry; one-dimensional, three-dimensional, or open-bound groups require exact custom-AABB handling when supported.*

Each term tightens a lower or upper bound along a feature axis. For a path over unique feature axes $(u_1, \dots, u_k)$, $k \leq 3$, the conjunction maps to

$$B_p = I_{p,1} \times \dots \times I_{p,k},$$

where $I_{p,j}$ may be lower-open or upper-closed depending on the original predicate. A row maps to

$$q_i = (X_{i,u_1}, \dots, X_{i,u_k}).$$

The desired semantic test is $z_{p,i} = 1$ if and only if $q_i \in B_p$ under the same open/closed comparison rules as the original path.

4 Scoring Without Dense Membership

The compact score path currently supports Pearson correlation and R^2 for decision-path indicators. Non-finite targets are excluded from every statistic. Let $V = \{i \mid y_i \text{ is finite}\}$ and $n_V = |V|$. For path p , define

$$S_x = \sum_{i \in V} z_{p,i}, \quad S_y = \sum_{i \in V} y_i, \quad S_{yy} = \sum_{i \in V} y_i^2, \quad S_{xy} = \sum_{i \in V} z_{p,i} y_i.$$

For $n_V > 0$, the centered terms are

$$C_{xx} = S_x - \frac{S_x^2}{n_V}, \quad C_{yy} = S_{yy} - \frac{S_y^2}{n_V}, \quad C_{xy} = S_{xy} - \frac{S_x S_y}{n_V}.$$

The Pearson score is

$$\rho_p = \begin{cases} \text{clip}(C_{xy} / \sqrt{C_{xx} C_{yy}}, -1, 1), & C_{xx} C_{yy} > 0, \\ 0, & \text{otherwise,} \end{cases}$$

and $R_p^2 = \text{clip}(\rho_p^2, 0, 1)$. When $n_V = 0$, the implementation returns zero for both metrics.

This formula shows why dense membership export is unnecessary. The score needs only per-path counts and target sums plus target-wide statistics. Therefore, the backend can evaluate membership transiently, reduce on device, and return one compact row per candidate with metric columns.

5 System Design

5.1 Ownership Boundary

The intended GAFIME v1 architecture uses a hard ownership split. Python exposes the public API; Rust owns validation, feature planning, scheduling, backend selection, fallback policy, and report ownership; CUDA owns the CUDA-specific execution boundary. The experimental ABI implements the backend half of this contract. The current public decision-path adapter is not yet wired to it.

- An approved Rust caller passes compact, validated decision-path descriptors and an explicit RT policy through the GPU C ABI.
- CUDA validates RT representability, builds geometry, launches OPTiX, and writes compact result rows.
- CUDA does not discover candidate paths, mutate scheduler policy, or infer Python behavior. A require-RT request fails explicitly if unavailable.
- First-hit requests return unsupported when their finite, bounded, two-dimensional, non-overlap proof cannot be established.

The explicit per-call policy is `DecisionPathRtPolicy::RequireRt`, encoded as `GAFIME_DECISION_PATH_FLAG_REQUIRE_RT` in the batch. It controls fallback only: an unavailable or unrepresentable RT request returns unsupported instead of executing the SM comparator. By contrast, the CUDA payload reads three process-global experimental test and profiling selectors:

- `GAFIME_CUDA_DECISION_PATH_RT`,
- `GAFIME_CUDA_DECISION_PATH_RT_SCORE`, and
- `GAFIME_CUDA_DECISION_PATH_RT_GEOMETRY`.

They are not a thread-local or per-call public policy ABI. The benchmark selects `firsthit` through the process environment and independently sets the per-call require-RT flag so an accidental SM fallback cannot produce a timing.

5.2 CUDA Source Split

The implementation keeps RT code out of the generic continuous CUDA kernels:

- `src/cuda/rt_kernels.cu` contains OptiX device programs, RT-specific CUDA kernels, point packing, exact hit filters, and direct-score accumulation helpers.
- `src/cuda/rt_launcher.cu` owns host-side RT planning, finite-box validation, OptiX pipeline setup, GAS/IAS construction, cached workspaces, grouped execution, and explicit comparator dispatch.

- `src/cuda/launcher.cu` remains the public C ABI bridge for decision-path RT symbols.

This source layout is part of the architectural contract. RT traversal logic is large enough to deserve explicit ownership; hiding it inside the generic CUDA score path would make later backend maintenance harder.

5.3 Grouped Instanced Launch

Candidate batches may contain many feature pairs. A single triangle GAS cannot interpret point coordinates for all feature-pair coordinate systems at once. The CUDA launcher groups paths by compatible feature axes, builds one triangle GAS per group, and wraps group GAS handles in one instance acceleration structure (IAS). Ray generation then launches

$$\text{launch shape} = (n, g, 1),$$

where g is the number of RT groups. The instance id selects the group-local path table, and a scatter map restores original path order before result export. This gives OPTIX a large batched launch without moving candidate discovery into the backend.

5.4 Score Modes

Bitset mode. OPTIX writes an idempotent device bitset, and a CUDA kernel reduces the bitset with the target vector. This is the conservative parity path because it avoids floating accumulation order in the traversal program.

Direct mode. The any-hit program updates per-path counts and target sums directly. This removes the temporary bitset and reduction scan but uses floating atomics for some statistics, so it is opt-in and documented with tolerance.

First-hit mode. For finite, bounded, two-dimensional groups where CUDA proves that boxes do not overlap, the any-hit program accepts the first exact hit and terminates traversal. This matches tree-leaf assignment: a row belongs to at most one leaf region in a partition.

Proposition 1 (First-hit equivalence). *Let $\mathcal{B} = \{B_1, \dots, B_m\}$ be a group of bounded two-dimensional decision regions such that the interiors are pairwise disjoint and the exact open/closed predicate guard assigns every boundary point to at most one region. For any row point q , first-hit traversal with the exact guard returns the same membership vector as evaluating all path predicates and selecting the unique inside region, if one exists.*

Proof. If q is outside all regions, no exact guard accepts a hit, so all membership values are zero. If q is inside one region B_j , disjointness and the boundary ownership guard imply no other region can also accept q . Terminating after the first accepted hit therefore records B_j and cannot suppress another valid region. The recorded membership is identical to exhaustive predicate evaluation. $\square$

First-hit mode is fail-closed. If the finite, bounded, 2D, non-overlapping proof is unavailable, the experimental first-hit selector returns unsupported rather than changing semantics or silently routing that call to the SM comparator. The independent require-RT ABI flag also forbids fallback.

RT support is also a build-time distribution choice. The default CUDA payload is RT-disabled and has no OptiX dependency. CMake build modes `off`, `on`, and `both` produce an RT-free payload, an RT-enabled payload, or distinct copies of each. Every measurement in this paper uses the

explicitly RT-enabled `libgafime_cuda_v1_rt.so`; it is not evidence about the standard RT-off artifact. The optional RT wheel is a separately selected GitHub Actions artifact, not a claimed PyPI release.

6 Numerical Policy

The correctness policy separates Boolean membership from floating score accumulation. Membership semantics are exact with respect to the GAFIME predicate model: **LE** means $x \leq t$, **GT** means $x > t$, and NaNs are not silently reinterpreted. The current upload records finiteness for the entire feature matrix, not only columns referenced by a path; any non-finite feature value makes the RT path unsupported. Non-finite target values do not make RT unsupported, but those rows are omitted from all Pearson/ R^2 statistics as defined above.

Path ids, result row ordering, and direct-mode hit accumulation in `uint32_t` remain integral under the RT row limit of `UINT32_MAX`. Score finalization is nevertheless single precision: the valid-target count is accumulated as `float`, and each direct hit count is converted to `float`. Therefore every integer count is guaranteed exactly representable only through 2^{24} ; larger counts are not covered by an exact-count claim. Both reported workloads use 262,144 rows and stay below this limit. Floating Pearson and R^2 values may also differ because of floating atomics or backend-specific reduction order. The present first-hit parity run has maximum absolute score difference 1.19209×10^{-7} against the CPU reference, below the documented spike tolerance of 10^{-4} .

7 Evaluation

7.1 Hardware and Workloads

The measurements in this disclosure were collected on a laptop system with an AMD Ryzen AI 9 HX 370 class CPU and an NVIDIA GeForce RTX 4060 Laptop GPU (Ada, `sm_89`, approximately 7.63 GiB reported global memory), CUDA 13.3, and OptiX 7.5 headers. The exact benchmark sources are:

```
tests/gpu/cuda_rt_membership_scale_bench.cpp
tests/release_measure/perf_05_cuda_rt_firsthit_scale.py
```

Complete build, execution, profiler, and PDF commands are recorded in:

```
docs/rt-gbdt-paper-repro.md
```

The implementation and harness files against which these claims were reviewed are pinned by:

```
docs/evidence/rt-gbdt-paper-source-evidence.sha256
```

Its own SHA-256 is:

```
fd3570bfffed1d7122dfe071b6b7fa5653d3b6e71e11637c937695b1dccc106bf
```

The immutable timing transcript is:

```
docs/evidence/rt-firsthit-sm89-timing.txt
```

Its SHA-256 is:

```
47e761b45dcbbc6d5a3c658939b34a3ad02d9837638a38393e034ce598b73d28
```

The reproduction note gives exact commands for validating those hashes and for replaying the captured configurations. The original benchmark executable hash was not retained, so the source manifest identifies the reviewed reproducibility state rather than proving a byte-identical capture binary. No performance benchmark was rerun for the release-readiness documentation review.

The synthetic workloads use bounded 2D boxes over feature-pair groups. The partitioned-grid mode creates non-overlapping boxes within each group, modeling the tree-leaf invariant required by first-hit traversal. We report all-pairs-equivalent work as

$$W_{\text{eq}} = n \times m,$$

where n is row count and m is path count. First-hit traversal does not execute $n \times m$ explicit predicate tests. Its actual launch contains $n \times g$ rays for g compatible geometry groups, and BVH traversal prunes the path set. Dividing $n \times g$ by the full timed score call produces an end-to-end effective ray rate; it includes launch and programmable CUDA work and is not an isolated RT-core counter. We report it beside W_{eq}/t .

Each hardened score run makes six calls after one resident matrix upload. The first timed call includes OptiX program initialization, geometry construction, and cache population. The next five calls reuse resident state; the reported warm number is the observed median sample, not a best-of- N value. The benchmark validates the worst parity error across the cold call and all five warm samples, so a correct median cannot hide an incorrect outlier. Warm results are steady-state timing evidence and must not be read as first-use latency.

7.2 Compact Output and Score Modes

Materializing an f32 path-major membership matrix for $1,048,576 \times 512$ requires 2.00 GiB. A byte mask requires 512 MiB, the conservative device bitset requires 64 MiB, and the final two-metric result requires only 4 KiB. Compact reduction is therefore part of the algorithm, not an output-format detail.

The RT bitset path retains a deterministic idempotent mask. Direct mode removes the mask scan and accumulates traversal statistics with floating atomics; it is experimental and uses the documented 10^{-4} tolerance. The hash-bound timing evidence below is limited to first-hit mode; this paper does not attach an unarchived standalone timing claim to the other score modes.

7.3 First-Hit Partitioned Score

The strongest result appears when the regions satisfy the tree-leaf-like non-overlap condition. Table 1 reports both end-to-end effective ray rate and all-pairs-equivalent work rate for eight geometry groups.

Table 1: First-hit partitioned scoring at 262,144 rows. Warm p50 uses five resident samples. The small case uses an exhaustive CPU reference; the large case uses the exact partition oracle.

Paths	Paths/group	First ms	Warm ms	G eff. ray/s	T eq/s	Max abs
8,192	1,024	56.109	0.886	2.367	2.424	1.19209×10^{-7}
1,048,576	131,072	467.746	20.180	0.104	13.621	5.58794×10^{-9}

The hash-bound timing transcript for the parity-covered case reported:

resident upload	6.696 ms,
CPU score reference	6128.597 ms (0.350 G eq/s),
first RT call	56.109 ms,
resident warm p50	0.886 ms (2.424 T eq/s),
maximum absolute difference	1.19209×10^{-7} .

The large case no longer skips correctness. Its oracle exploits the known partition structure: for each row and group it identifies the unique cell using the same floating open-lower/closed-upper boundaries, accumulates that path’s statistics, and finalizes every path. This is $O(n_g + m)$ rather than an exhaustive $O(nm)$ scan. The benchmark also rejects non-finite scores and checks result row count, order, family, rank, and candidate id on every repetition.

7.4 Profiler Evidence

A separate Nsight Compute 2026.2.1 full-set capture profiled one $65,536 \times 8,192$ first-hit call. Full replay perturbs end-to-end timing, so these durations are used only to identify hot units. PerfDigest reduced the report without copying the raw counter stream into the review context. The exact 31,848,275-byte capture, docs/evidence/rt-firsthit-sm89-65536x8192-final.ncu-rep, has SHA-256 5461bf86495d9a12666891bba2f334ecea8b16b3c8cb806168a557101a52c331. The reproduction note pins the exact PerfDigest calls; the digest is not taken from the similarly named earlier review capture. The resident query contained an `optixLaunch` at 196.992 μ s, grouped point packing at 25.088 μ s, and score scatter/finalization at 4.480 μ s. Eight group GAS builds were approximately 156 μ s each and the IAS build was 49.952 μ s; those builds belong to first-use setup.

For `optixLaunch`, the available counters report 24.932% compute-pipe peak, 10.878% DRAM peak, 54.223% achieved occupancy, 53.408% L1 hit, and 96.864% L2 hit, with 72 registers/thread. This supports a traversal-dominated, cache-resident interpretation relative to packing and scatter. It does not measure RT-core saturation directly, and the unavailable fixed-function counter must not be inferred from SM compute or branch metrics.

7.5 Interpretation

The first-hit result is large because the workload matches both the BVH algorithm and the traversal hardware:

- path candidates are bounded regions, not arbitrary branch programs;
- the partitioned-grid invariant lets each row-group ray terminate after the first exact hit;
- grouped instancing creates a large launch rather than many tiny launches;
- score output is compact and does not copy dense membership to host;
- resident sessions reuse geometry, packed points, target statistics, and scratch buffers when inputs are unchanged.

Dense overlapping boxes do not show the same behavior. The result therefore cannot be interpreted as a general RT-core speedup over exhaustive CUDA. The first-hit method changes the algorithm from all-pairs testing to BVH search with at most one accepted hit per row-group ray. A fair attribution study still needs a CUDA baseline that exploits the same partition structure without OptiX.

8 Limitations

The implementation is intentionally narrow. The fastest first-hit path currently applies only to finite, bounded, non-overlapping 2D groups. The compact RT decision-path score ABI supports Pearson and R^2 in this spike; MI and Spearman require additional parity work. CUDA/OptiX is the only RT backend. ROCm and Metal do not expose this RT-core path through the current GAFIME ABI.

The public Python decision-path workflow does not yet invoke the compact RT score ABI. It still materializes candidate membership before generic continuous analysis. The paper therefore makes no end-to-end GAFIME speedup claim. The default CUDA distribution is also RT-disabled; users must select an RT-enabled payload explicitly.

Both evaluated workloads are synthetic partition grids. The large case is validated by a structure-aware CPU oracle rather than exhaustive all-pairs CPU evaluation. No production ensemble paths, training loop, inference service, or feature-mining search were benchmarked. The current SM comparator evaluates all row-path pairs and is not an algorithmically matched partition index, so it cannot isolate the contribution of fixed-function traversal hardware.

The evaluation uses one Ada laptop GPU. Larger desktop RTX devices or different generations may shift the bottleneck toward acceleration-structure size, target-stat atomic pressure, memory bandwidth, or launch scheduling. Available Nsight Compute counters did not expose a direct RT-core saturation percentage. First-call latency is tens to hundreds of milliseconds because it includes program and acceleration-structure setup; resident warm timing applies only when unchanged geometry and buffers can be reused.

9 Future Work

Several pieces remain before this becomes a broader GBDT acceleration method:

1. wire the compact ABI through the Rust-owned public decision-path planner without candidate-expanded row materialization;
2. evaluate paths extracted from real trained ensembles, not only synthetic partition grids;
3. compare against a CUDA partition index with the same first-hit algorithmic advantage, then separate build, query, memory, and energy costs;
4. add deterministic buffered first-hit statistics and compare them with direct floating atomics;
5. extend compact RT scoring to MI and Spearman only after estimator and significance parity are proven;
6. test Turing, Ampere, Ada, Hopper, and Blackwell devices and report acceleration-structure memory explicitly;
7. compare against production tree-inference and GPU-GBDT systems while preserving the distinction between candidate scoring, inference, and training.

10 Conclusion

This paper reports a systems mapping from bounded decision-path scoring to hardware ray tracing traversal and on-device fused reduction. The key insight is not that every tree operation is geometric.

The useful subset is narrower: finite low-dimensional path regions can be scored as point-in-region tests, and tree-leaf-like non-overlap makes first-hit traversal semantically valid.

The implementation establishes a compact low-level ABI and fail-closed representability rules, but public-path integration remains future work. On the parity-covered RTX 4060 Laptop case, resident warm performance is 2.424 T membership-equivalent evaluations/s at 2.367 G end-to-end effective rays/s with maximum absolute score error 1.19209×10^{-7} . On the larger oracle-validated case it is 13.621 T membership-equivalent evaluations/s at 0.104 G end-to-end effective rays/s with 5.58794×10^{-9} error. The differing rates emphasize the central conclusion: this is a promising combination of tree-like partition structure, BVH search, RT-assisted traversal, and compact CUDA reduction, not evidence that RT cores accelerate arbitrary GBDT computation.

References

- [1] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. <https://doi.org/10.1214/aos/1013203451>.
- [2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. <https://doi.org/10.1145/2939672.2939785>.
- [3] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 30, 2017. <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>.
- [4] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 31, 2018. <https://papers.nips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features>.
- [5] Zeyi Wen, Hanfeng Liu, Jiashuai Shi, Qinbin Li, Bingsheng He, and Jian Chen. ThunderGBM: Fast GBDTs and random forests on GPUs. *Journal of Machine Learning Research*, 21(108):1–5, 2020. <https://www.jmlr.org/papers/v21/19-095.html>.
- [6] Huan Zhang, Si Si, and Cho-Jui Hsieh. GPU-acceleration for large-scale tree boosting. arXiv:1706.08359, 2017. <https://arxiv.org/abs/1706.08359>.
- [7] Hyunsu Cho and Mu Li. Treelite: Toolbox for decision tree deployment. In *Systems for Machine Learning*, 2018. <https://www.amazon.science/publications/treelite-toolbox-for-decision-tree-deployment>.
- [8] NVIDIA RAPIDS. Forest Inference Library documentation. <https://docs.rapids.ai/api/cuml/stable/fil/>.
- [9] Ashwin Prasad, Sampath Rajendra, Kaushik Rajan, R. Govindarajan, and Uday Bondhugula. An optimizing compiler for decision tree based ML inference. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture*, 2022. <https://doi.org/10.1109/MICRO56248.2022.00043>.

- [10] Stefan Zellmann, Daniel Seifried, Nate Morrical, Ingo Wald, Will Usher, Jamie A. P. Law-Smith, Stefanie Walch-Gassner, and Andre Hinkenjann. Point containment queries on ray tracing cores for AMR flow visualization. *arXiv:2202.12020*, 2022. <https://arxiv.org/abs/2202.12020>.
- [11] Justus Henneberg and Felix Schuhknecht. RTIndex: Exploiting hardware-accelerated GPU raytracing for database indexing. *Proceedings of the VLDB Endowment*, 16(13):4268–4281, 2023. <https://doi.org/10.14778/3625054.3625063>.
- [12] Enzo Meneses, Cristobal A. Navarro, Hector Ferrada, and Felipe A. Quezada. Accelerating range minimum queries with ray tracing cores. *arXiv:2306.03282*, 2023. <https://arxiv.org/abs/2306.03282>.
- [13] Yangming Lv, Kai Zhang, Ziming Wang, Xiaodong Zhang, Rubao Lee, Zhenying He, Yinan Jing, and X. Sean Wang. RTScan: Efficient scan with ray tracing cores. *Proceedings of the VLDB Endowment*, 17(6):1460–1472, 2024. <https://doi.org/10.14778/3648160.3648183>.
- [14] Harish Doraiswamy and Jayant R. Haritsa. Raster is faster: Rethinking ray tracing in database indexing. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2026. <https://www.vldb.org/cidrdb/papers/2026/p18-doraiswamy.pdf>.
- [15] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. OptiX: A general purpose ray tracing engine. *ACM Transactions on Graphics*, 29(4), 2010. <https://doi.org/10.1145/1778765.1778803>.
- [16] NVIDIA. NVIDIA Turing GPU Architecture. White paper, 2018. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>.
- [17] Timo Aila and Samuli Laine. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of High Performance Graphics*, 2009. <https://users.aalto.fi/~ailat1/HPG2009/>.
- [18] Timo Aila, Samuli Laine, and Tero Karras. Understanding the efficiency of ray traversal on GPUs: Kepler and Fermi addendum. NVIDIA Technical Report NVR-2012-02, 2012. https://research.nvidia.com/publication/2012-06_understanding-efficiency-ray-traversal-gpus-kepler-and-fermi-addendum.
- [19] Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, and Manfred Ernst. Embree: A kernel framework for efficient CPU ray tracing. *ACM Transactions on Graphics*, 33(4), 2014. <https://doi.org/10.1145/2601097.2601199>.